Lab 5: Static Code Analysis Report

Name: Samarth Rajesh Rao
SRN: PES1UG24AM244
Assigned repo: L5_06_it_helpdesk (IT Helpdesk Ticket Allocator)
File analysed: helpdesk_system.py
Date: 27 October 2026

1. Objective

Use Pylint, Bandit and Flake8 to find quality, security and style problems in the assigned script, fix them, and verify the fixes by re-running the tools.

2. Summary: before and after

Tool	Before	After
Pylint	5.53 / 10 (21 messages)	10.00 / 10 (0 messages)
Bandit	4 issues: 1 High, 3 Low	0 issues (3 justified nosec suppressions)
Flake8	15 warnings (E302, E501, E305, E711, E722, F841)	0 warnings

3. Commits

Commit	Tool	What was fixed
1	Bandit	B602 shell=True, B105 hardcoded password, B110 try/except/pass, B404
2	Flake8	E302, E305, E501, E711, F841
3	Pylint	Docstrings, naming, global, mutable default, with-open, f-strings, input validation, __main__ guard; after reports

4. Issues table

Issue	Type	Line(s)	Description	Fix approach
Shell injection, shell=True (Bandit B602, High)	Security	38	'ping -c 1 ' + hostname run via shell; a hostname like 'x; rm -rf ~' runs arbitrary commands. -c also fails on Windows.	Validate hostname with regex; subprocess.call([...]) with no shell; -n/-c by platform
Hard-coded password (B105)	Security	8, 55	ADMIN_PASSWORD literal in source and repeated in main(); compared with ==	Read from HELPDESK_ADMIN_PASSWORD env var; hmac.compare_digest; deny if unset
Bare except: pass (B110, W0702, E722)	Bug/Security	21-24	add_note hides all errors; a note for a missing ticket is lost	Explicit existence check, message and return False
Mutable default arg (W0102)	Bug	10	audit_trail=[] shared across all calls	Default None; create list inside function
Unvalidated urgency (not flagged by tools)	Logic bug	13, 52-54	-7 or 99 accepted as priority; 'abc' crashes with a traceback	parse_priority() enforces 1-5; main() catches ValueError

KeyError on missing ticket	Logic bug	17-18, 26-28	assign_ticket/close_ticket crash on unknown id; wrong password fails silently	Existence checks; return True/False
== None (E711, C0121)	Style	33	Comparison to None with ==	Use 'is None'
global statement (W0603, C0103)	Design	7, 11	Mutable global counter rebound via global	itertools.count(101) and module-level TICKETS
camelCase name (C0103), C0206	Style	30	unresolvedHighPriority	Renamed unresolved_high_priority; comprehension over .items()
open() without with/encoding (R1732, W1514)	Resource	40-43	Handle leaks on error; no encoding	with open(..., encoding='utf-8'); json.dump
PEP 8 layout (E302, E305, E501)	Style	10-58	Missing blank lines; 3 long lines	Reformatted
Unused variable (F841, W0612)	Style	47	t2 never used	Removed binding
Missing docstrings (C0114, C0116)	Docs	all	No docstrings	Added
% formatting (C0209)	Style	14	Old-style formatting	f-strings
No __main__ guard	Design	58	main() runs on import	if __name__ == '__main__'
import subprocess (B404, Low)	Security	2	Informational	Kept; nosec B404 (and B603/B607 on the safe call)

5. Tool reports: before

pylint_report.txt (before)

```
************* Module helpdesk_system
helpdesk_system.py:13:0: C0301: Line too long (118/100) (line-too-long)
helpdesk_system.py:1:0: C0114: Missing module docstring (missing-module-docstring)
helpdesk_system.py:7:0: C0103: Constant name "ticket_counter" doesn't conform to UPPER_CASE naming style
(invalid-name)
helpdesk_system.py:10:0: C0116: Missing function or method docstring (missing-function-docstring)
helpdesk_system.py:10:0: W0102: Dangerous default value [] as argument (dangerous-default-value)
helpdesk_system.py:11:4: W0603: Using the global statement (global-statement)
helpdesk_system.py:14:23: C0209: Formatting a regular string which could be an f-string (consider-using-f-
string)
helpdesk_system.py:17:0: C0116: Missing function or method docstring (missing-function-docstring)
helpdesk_system.py:20:0: C0116: Missing function or method docstring (missing-function-docstring)
helpdesk_system.py:23:4: W0702: No exception type(s) specified (bare-except)
helpdesk_system.py:26:0: C0116: Missing function or method docstring (missing-function-docstring)
helpdesk_system.py:30:0: C0116: Missing function or method docstring (missing-function-docstring)
helpdesk_system.py:30:0: C0103: Function name "unresolvedHighPriority" doesn't conform to snake_case naming
style (invalid-name)
helpdesk_system.py:32:4: C0206: Consider iterating with .items() (consider-using-dict-items)
helpdesk_system.py:33:43: C0121: Comparison 'tickets[t]['tech'] == None' should be 'tickets[t]['tech'] is
None' (singleton-comparison)
helpdesk_system.py:37:0: C0116: Missing function or method docstring (missing-function-docstring)
helpdesk_system.py:40:0: C0116: Missing function or method docstring (missing-function-docstring)
helpdesk_system.py:41:8: W1514: Using open without explicitly specifying an encoding (unspecified-encoding)
helpdesk_system.py:41:8: R1732: Consider using 'with' for resource-allocating operations (consider-using-
with)
helpdesk_system.py:45:0: C0116: Missing function or method docstring (missing-function-docstring)
helpdesk_system.py:47:4: W0612: Unused variable 't2' (unused-variable)
```

```
----------------------------------
Your code has been rated at 5.53/10
```

bandit_report.txt (before)

```
Run started:2026-10-27 06:28:39.706906+00:00

Test results:
>> Issue: [B404:blacklist] Consider possible security implications associated with the subprocess module.
   Severity: Low   Confidence: High
   CWE: CWE-78 (https://cwe.mitre.org/data/definitions/78.html)
   More Info: https://bandit.readthedocs.io/en/1.9.4/blacklists/blacklist_imports.html#b404-import-subprocess
   Location: .\helpdesk_system.py:2:0
1       import json
2       import subprocess
3       from datetime import datetime

--------------------------------------------------
>> Issue: [B105:hardcoded_password_string] Possible hardcoded password: 'helpdesk@123'
   Severity: Low   Confidence: Medium
   CWE: CWE-259 (https://cwe.mitre.org/data/definitions/259.html)
   More Info: https://bandit.readthedocs.io/en/1.9.4/plugins/b105_hardcoded_password_string.html
   Location: .\helpdesk_system.py:8:17
7       ticket_counter = 100
8       ADMIN_PASSWORD = "helpdesk@123"
9

--------------------------------------------------
>> Issue: [B110:try_except_pass] Try, Except, Pass detected.
   Severity: Low   Confidence: High
   CWE: CWE-703 (https://cwe.mitre.org/data/definitions/703.html)
   More Info: https://bandit.readthedocs.io/en/1.9.4/plugins/b110_try_except_pass.html
   Location: .\helpdesk_system.py:23:4
22              tickets[ticket_id]["notes"].append(note)
23          except:
24              pass
25

--------------------------------------------------
>> Issue: [B602:subprocess_popen_with_shell_equals_true] subprocess call with shell=True identified, security
issue.
   Severity: High   Confidence: High
   CWE: CWE-78 (https://cwe.mitre.org/data/definitions/78.html)
   More Info:
https://bandit.readthedocs.io/en/1.9.4/plugins/b602_subprocess_popen_with_shell_equals_true.html
   Location: .\helpdesk_system.py:38:11
37      def ping_user_machine(hostname):
38          return subprocess.call("ping -c 1 " + hostname, shell=True)
39

--------------------------------------------------

Code scanned:
        Total lines of code: 47
        Total lines skipped (#nosec): 0
        Total potential issues skipped due to specifically being disabled (e.g., #nosec BXXX): 0

Run metrics:
        Total issues (by severity):
                Undefined: 0
```

```
                Low: 3
                Medium: 0
                High: 1
        Total issues (by confidence):
                Undefined: 0
                Low: 0
                Medium: 1
                High: 3
Files skipped (0):
```

flake8_report.txt (before)

```
helpdesk_system.py:10:1: E302 expected 2 blank lines, found 1
helpdesk_system.py:13:80: E501 line too long (118 > 79 characters)
helpdesk_system.py:14:80: E501 line too long (86 > 79 characters)
helpdesk_system.py:17:1: E302 expected 2 blank lines, found 1
helpdesk_system.py:20:1: E302 expected 2 blank lines, found 1
helpdesk_system.py:23:5: E722 do not use bare 'except'
helpdesk_system.py:26:1: E302 expected 2 blank lines, found 1
helpdesk_system.py:30:1: E302 expected 2 blank lines, found 1
helpdesk_system.py:33:63: E711 comparison to None should be 'if cond is None:'
helpdesk_system.py:37:1: E302 expected 2 blank lines, found 1
helpdesk_system.py:40:1: E302 expected 2 blank lines, found 1
helpdesk_system.py:45:1: E302 expected 2 blank lines, found 1
helpdesk_system.py:47:5: F841 local variable 't2' is assigned to but never used
helpdesk_system.py:52:80: E501 line too long (80 > 79 characters)
helpdesk_system.py:58:1: E305 expected 2 blank lines after class or function definition, found 1
```

6. Tool reports: after

pylint_report.txt (after)

```
--------------------------------------------------------------------
Your code has been rated at 10.00/10 (previous run: 9.87/10, +0.13)
```

bandit_report.txt (after)

```
Run started:2026-10-27 06:30:09.642452+00:00

Test results:
        No issues identified.

Code scanned:
        Total lines of code: 109
        Total lines skipped (#nosec): 0
        Total potential issues skipped due to specifically being disabled (e.g., #nosec BXXX): 3

Run metrics:
        Total issues (by severity):
                Undefined: 0
                Low: 0
                Medium: 0
                High: 0
        Total issues (by confidence):
                Undefined: 0
                Low: 0
                Medium: 0
```

```
               High: 0
Files skipped (0):
```

flake8_report.txt (after)

```
(no output - no warnings)
```

7. Source code

Before (original helpdesk_system.py)

```
import json
import subprocess
from datetime import datetime

# Global state
tickets = {}
ticket_counter = 100
ADMIN_PASSWORD = "helpdesk@123"

def raise_ticket(user, issue_type, urgency="3", audit_trail=[]):
    global ticket_counter
    ticket_counter += 1
    tickets[ticket_counter] = {"user": user, "issue": issue_type, "priority": int(urgency), "tech": None,
"notes": []}
    audit_trail.append("%s raised #%d at %s" % (user, ticket_counter, datetime.now()))
    return ticket_counter

def assign_ticket(ticket_id, technician):
    tickets[ticket_id]["tech"] = technician

def add_note(ticket_id, note):
    try:
        tickets[ticket_id]["notes"].append(note)
    except:
        pass

def close_ticket(ticket_id, password):
    if password == ADMIN_PASSWORD:
        del tickets[ticket_id]

def unresolvedHighPriority():
    result = []
    for t in tickets:
        if tickets[t]["priority"] >= 4 and tickets[t]["tech"] == None:
            result.append(t)
    return result

def ping_user_machine(hostname):
    return subprocess.call("ping -c 1 " + hostname, shell=True)

def save_tickets(file="tickets.json"):
    f = open(file, "w")
    f.write(json.dumps(tickets))
    f.close()

def main():
    t1 = raise_ticket("anil", "printer", "5")
```

```
    t2 = raise_ticket("meera", "wifi", "4")
    t3 = raise_ticket("joe", "vpn", "-7")
    assign_ticket(t1, "Tech-Kiran")
    add_note(999, "Called user")
    print("Unassigned high priority:", unresolvedHighPriority())
    level = input("Urgency for a new laptop ticket (1-5, Enter for 3): ") or "3"
    raise_ticket("sana", "laptop", level)
    close_ticket(t3, "helpdesk@123")
    save_tickets()
    print("Open tickets:", list(tickets.keys()))

main()
```

After (updated helpdesk_system.py)

```
"""IT helpdesk ticket allocator.

Raises tickets with an urgency level, assigns technicians, records notes,
lists unassigned high-priority tickets and saves everything to a JSON file.
"""
import hmac
import itertools
import json
import os
import platform
import re
import subprocess  # nosec B404
from datetime import datetime

MIN_PRIORITY = 1
MAX_PRIORITY = 5
DEFAULT_PRIORITY = 3
HIGH_PRIORITY = 4

# Read from the environment instead of hard-coding the secret in source
ADMIN_PASSWORD = os.environ.get("HELPDESK_ADMIN_PASSWORD", "")
HOSTNAME_PATTERN = re.compile(
    r"^[A-Za-z0-9]([A-Za-z0-9.-]{0,251}[A-Za-z0-9])?$")


TICKETS = {}
_ticket_ids = itertools.count(101)


def parse_priority(urgency):
    """Convert urgency to an int and make sure it is within 1-5."""
    try:
        priority = int(urgency)
    except (TypeError, ValueError) as exc:
        raise ValueError(f"Urgency must be a number, got {urgency!r}") from exc
    if not MIN_PRIORITY <= priority <= MAX_PRIORITY:
        raise ValueError(
            f"Urgency must be between {MIN_PRIORITY} and {MAX_PRIORITY}, "
            f"got {priority}")
    return priority


def raise_ticket(user, issue_type, urgency=str(DEFAULT_PRIORITY),
                 audit_trail=None):
    """Create a ticket and return its id.
```

```
    A fresh audit trail is used per call unless one is passed in.
    """
    priority = parse_priority(urgency)
    if audit_trail is None:
        audit_trail = []
    ticket_id = next(_ticket_ids)
    TICKETS[ticket_id] = {
        "user": user,
        "issue": issue_type,
        "priority": priority,
        "tech": None,
        "notes": [],
    }
    audit_trail.append(
        f"{user} raised #{ticket_id} at {datetime.now()}")
    return ticket_id


def assign_ticket(ticket_id, technician):
    """Assign a technician; return False if the ticket does not exist."""
    if ticket_id not in TICKETS:
        return False
    TICKETS[ticket_id]["tech"] = technician
    return True


def add_note(ticket_id, note):
    """Append a note; return False if the ticket does not exist."""
    if ticket_id not in TICKETS:
        print(f"Cannot add note: ticket #{ticket_id} does not exist")
        return False
    TICKETS[ticket_id]["notes"].append(note)
    return True


def close_ticket(ticket_id, password):
    """Delete a ticket if the admin password matches."""
    if not ADMIN_PASSWORD or not hmac.compare_digest(
            password.encode(), ADMIN_PASSWORD.encode()):
        return False
    if ticket_id not in TICKETS:
        return False
    del TICKETS[ticket_id]
    return True


def unresolved_high_priority():
    """Return ids of high-priority tickets with no technician assigned."""
    return [
        ticket_id for ticket_id, ticket in TICKETS.items()
        if ticket["priority"] >= HIGH_PRIORITY and ticket["tech"] is None
    ]


def ping_user_machine(hostname):
    """Ping a host once and return the ping exit code."""
    if not HOSTNAME_PATTERN.match(hostname):
        raise ValueError(f"Invalid hostname: {hostname!r}")
    count_flag = "-n" if platform.system() == "Windows" else "-c"
```

```
    # Argument list with no shell: hostname cannot inject extra commands
    return subprocess.call(  # nosec B603 B607
        ["ping", count_flag, "1", hostname])


def save_tickets(file="tickets.json"):
    """Write all tickets to a JSON file."""
    with open(file, "w", encoding="utf-8") as handle:
        json.dump(TICKETS, handle)


def main():
    """Demo run of the helpdesk workflow."""
    t1 = raise_ticket("anil", "printer", "5")
    raise_ticket("meera", "wifi", "4")
    t3 = raise_ticket("joe", "vpn", "3")
    assign_ticket(t1, "Tech-Kiran")
    add_note(999, "Called user")
    print("Unassigned high priority:", unresolved_high_priority())
    level = input(
        "Urgency for a new laptop ticket (1-5, Enter for 3): ") or "3"
    try:
        raise_ticket("sana", "laptop", level)
    except ValueError as err:
        print("Ticket not created:", err)
    close_ticket(t3, ADMIN_PASSWORD)
    save_tickets()
    print("Open tickets:", list(TICKETS))


if __name__ == "__main__":
    main()
```

8. Reflection questions

Q1. Which issues were the easiest to fix and which were the hardest?

The easiest were mechanical ones: == None to is None, blank lines, long lines, an unused variable and missing docstrings. Flake8 states the exact fix and nothing else depends on them. The hardest needed a decision about behaviour. Replacing shell=True meant validating the hostname, building an argument list and handling the platform difference for ping flags. Removing the hard-coded password meant deciding where the secret should come from and what happens when it is missing. Removing the global counter meant reshaping how ticket ids are generated.

Q2. Did the tools report any false positives?

Mostly no, but some findings are low value. Bandit B404 only says subprocess was imported, and after the shell was removed B603/B607 still fire on a call that is safe because the arguments are a validated list; I suppressed these with targeted nosec comments. B105 was a true positive here but would be a false positive for any variable that merely has 'password' in its name. Pylint's constant-naming complaint about ticket_counter was a style opinion rather than a defect. The tools also missed real bugs: priority -7 was accepted, non-numeric input crashed the program and unknown ticket ids raised KeyError. I found those by running the program.

Q3. How would you integrate static analysis into a real workflow?

Run flake8, pylint and bandit locally through pre-commit hooks so problems are caught before a commit, and again in CI (for example GitHub Actions) on every pull request. Fail the build on any Bandit finding, any Flake8 warning or a Pylint score under a threshold such as 9.5. Keep configuration (.flake8, .pylintrc, pyproject.toml) in the repo so everyone uses the same rules, and require a justification for every nosec or pylint disable.

Q4. What tangible improvements did you see in the code?

Pylint rose from 5.53 to 10.00, Flake8 from 15 warnings to 0 and Bandit from 1 High and 3 Low issues to 0. The behaviour is also safer and more robust: no command injection, no secret in source, invalid urgency and missing tickets are reported instead of crashing or being ignored, the audit trail no longer leaks between calls and the file handle is always closed. Docstrings, snake_case names and the __main__ guard make the code easier to read and import.